

Universal Payments Engineering Lab

Manual navegable de 28 modalidades de pago

Desde el inicio de una orden hasta confirmación, ledger, fallos, seguridad y conciliación.

Markdown	Fuente única revisable en GitHub
HTML	Sitio generado para navegar y visualizar Mermaid
PDF	Manual descargable con índice y enlaces internos

[Abrir explicación de formatos en GitHub Pages](#)

Índice de casos

Selecciona un caso. El enlace salta dentro de este PDF; no abre otra aplicación.

- | | |
|------------------------------------|--|
| 01. Efectivo y caja | 02. Cheques y órdenes en papel |
| 03. Tarjetas | 04. Transbank Webpay Plus |
| 05. Transbank Oneclick Mall | 06. Khipu |
| 07. Mercado Pago | 08. POS, mPOS y SoftPOS |
| 09. Wallets tokenizadas | 10. Saldo almacenado y gift cards |
| 11. Mobile money | 12. Pagos QR |
| 13. Transferencia bancaria | 14. ACH y transferencias batch |
| 15. Débito directo | 16. Pagos instantáneos |
| 17. Links y solicitud de pago | 18. Voucher pagable en efectivo |
| 19. BNPL y crédito alternativo | 20. Cobro mediante operador móvil |
| 21. Marketplaces, splits y payouts | 22. Pagos B2B y tesorería |
| 23. Pagos internacionales | 24. RTGS y alto valor |
| 25. Open Finance e iniciación | 26. Bitcoin, Lightning y activos digitales |
| 27. Pagos máquina a máquina | 28. Pagos realizados por agentes |

Cómo leer cada caso

Cada capítulo conserva el mismo orden: necesidad, ejemplo, recorrido, fallo, implementación, configuración, seguridad, pruebas y checklist LIVE. El diagrama aparece en todos los casos y no depende de Mermaid ni de conexión a Internet.

01. Efectivo y caja

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

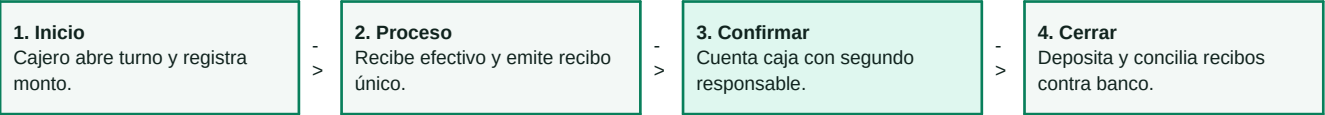
Sirve para: Cobro presencial sin red electrónica.

Modelo mental: El dinero cambia de manos ahora, pero la evidencia digital y el depósito aparecen después.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Una cafetería cobra \$12.500 en efectivo. Al cierre, el recibo, la caja física y el depósito del día siguiente deben explicar esos mismos \$12.500.	Cajero abre turno y registra monto. Recibe efectivo y emite recibo único.	Cuenta caja con segundo responsable. Después: Deposita y concilia recibos contra banco.

Figura 1. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Cajero abre turno y registra monto.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Recibe efectivo y emite recibo único.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Cuenta caja con segundo responsable.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Deposita y concilia recibos contra banco.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Faltante/sobrante queda como excepción; nunca se borra.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Registrar apertura de caja y responsable.
- Emitir recibo por cada recepción.
- Cerrar, depositar y conciliar diferencia por turno.

Evidencia de éxito: Recibo + arqueo + comprobante de depósito asociados a la misma referencia.

Construcción concreta: Caja web + roles + impresora + PostgreSQL + archivo bancario.

Capa	Tecnología / responsabilidad
Frontend	Interfaz web administrativa para registrar recepción, custodia y diferencias; no inicia una red de pagos.
Backend	Python/FastAPI o el stack transaccional existente del comercio.
API	API interna REST/JSON y, cuando exista, archivo/API de banco o red externa.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL con auditoría inmutable y ledger de doble entrada.
Operación	Control de acceso por rol, doble aprobación, respaldos y conciliación diaria.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Efectivo y caja solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- OWASP · Application Security Verification Standard
- PCI SSC · PCI DSS

02. Cheques y órdenes en papel

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

Sirve para: Cheque u orden física con cobro diferido.

Modelo mental: Recibir un cheque no equivale a cobrarlo: existe un período de presentación y posible devolución.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Una empresa recibe un cheque por \$1.200.000. La factura sigue pendiente hasta que el banco confirme el cobro; recibir el papel no libera el pedido.	Registra emisor, monto, fecha e imagen restringida. Entrega el documento al banco y queda PENDING.	El banco informa cobro o devolución. Después: Aplica asiento final y concilia cartola.

Figura 2. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Registra emisor, monto, fecha e imagen restringida.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Entrega el documento al banco y queda PENDING.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	El banco informa cobro o devolución.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Aplica asiento final y concilia cartola.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Cheque devuelto revierte disponibilidad y abre cobranza.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Capturar referencia sin almacenar imágenes innecesarias.
- Mantener estado pendiente durante clearing.
- Aplicar abono o devolución mediante evento bancario.

Evidencia de éxito: Confirmación del banco y ausencia de devolución dentro de la ventana aplicable.

Construcción concreta: Backoffice + custodia + API/archivo bancario + reglas locales.

Capa	Tecnología / responsabilidad
Frontend	Interfaz web administrativa para registrar recepción, custodia y diferencias; no inicia una red de pagos.
Backend	Python/FastAPI o el stack transaccional existente del comercio.
API	API interna REST/JSON y, cuando exista, archivo/API de banco o red externa.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL con auditoría inmutable y ledger de doble entrada.
Operación	Control de acceso por rol, doble aprobación, respaldos y conciliación diaria.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Cheques y órdenes en papel solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- ISO · ISO 20022
- BIS CPMI · sistemas de pago

03. Tarjetas

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

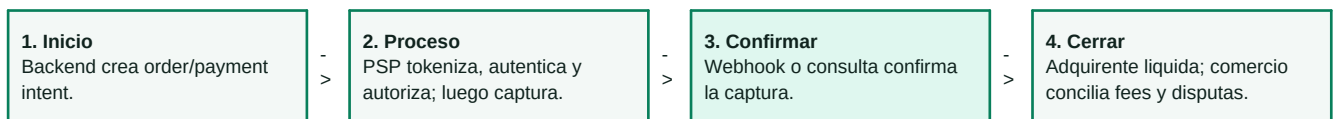
Sirve para: Pago inmediato con tarjeta física o digital.

Modelo mental: La tarjeta autoriza un cargo; captura, liquidación y chargeback son hechos posteriores.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Una tienda vende un notebook por \$649.990. El cliente ve una aprobación inmediata, pero el comercio todavía debe capturar, liquidar y conciliar el abono neto.	Backend crea order/payment intent. PSP tokeniza, autentica y autoriza; luego captura.	Webhook o consulta confirma la captura. Después: Adquirente liquida; comercio concilia fees y disputas.

Figura 3. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Backend crea order/payment intent.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	PSP tokeniza, autentica y autoriza; luego captura.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Webhook o consulta confirma la captura.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Adquirente liquida; comercio concilia fees y disputas.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Timeout queda UNKNOWN; chargeback se registra por separado.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Elegir checkout alojado o tokenización.
- Crear intent y autenticar cuando corresponda.
- Confirmar por webhook, contabilizar y conciliar settlement.

Evidencia de éxito: ID del proveedor, estado capturado, evento verificado y línea de settlement.

Construcción concreta: Checkout alojado + REST API + webhook + PostgreSQL + PCI DSS.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Tarjetas solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- PCI SSC · PCI DSS
- EMVCo · Especificaciones

04. Transbank Webpay Plus

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

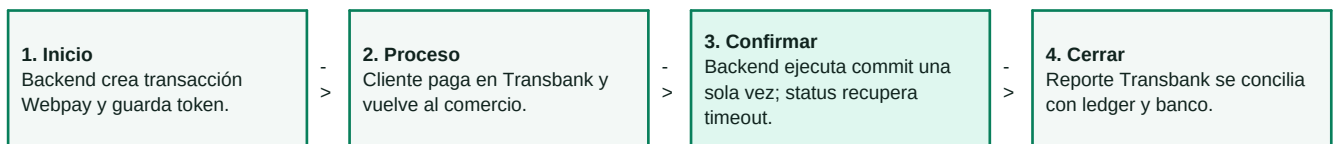
Sirve para: Tarjetas en Chile mediante checkout alojado.

Modelo mental: Tu web crea una transacción, el cliente paga en Transbank y tu backend confirma el token al retornar.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Una persona compra entradas por \$19.990. Tu backend crea la transacción, Transbank procesa la tarjeta y sólo el commit del backend permite confirmar la orden.	Backend crea transacción Webpay y guarda token. Cliente paga en Transbank y vuelve al comercio.	Backend ejecuta commit una sola vez; status recupera timeout. Después: Reporte Transbank se concilia con ledger y banco.

Figura 4. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Backend crea transacción Webpay y guarda token.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Cliente paga en Transbank y vuelve al comercio.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Backend ejecuta commit una sola vez; status recupera timeout.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Reporte Transbank se concilia con ledger y banco.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Retorno sin commit no prueba pago; UNKNOWN se consulta.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Crear transacción REST en backend.
- Redirigir al formulario alojado.
- Ejecutar commit una sola vez y consultar ante timeout.

Evidencia de éxito: Token confirmado por commit/status y conciliado con el reporte del comercio.

Construcción concreta: Contrato Transbank + REST Webpay + callback + credenciales separadas.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.

Capa	Tecnología / responsabilidad
API	REST Webpay Plus: create → redirección alojada → commit en backend → status/refund.
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Crear la integración con credenciales de integración de Transbank; para LIVE, contratar Webpay y completar puesta en producción.

Costo: El laboratorio cuesta \$0. La tarifa real depende del contrato del comercio: consultar la oferta vigente de Transbank antes de decidir.

Decisión: Usa Transbank Webpay Plus solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.
TRANSBANK_COMMERCE_CODE	Sí	No	Identificar el comercio Webpay.
TRANSBANK_API_KEY	Sí	Sí	Autenticar la API Webpay.
TRANSBANK_BASE_URL	Sí	No	Separar explícitamente integración y producción.

Callbacks: /payments/webpay/return

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- Transbank · Cómo empezar
- Transbank · API Webpay

05. Transbank Oneclick Mall

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

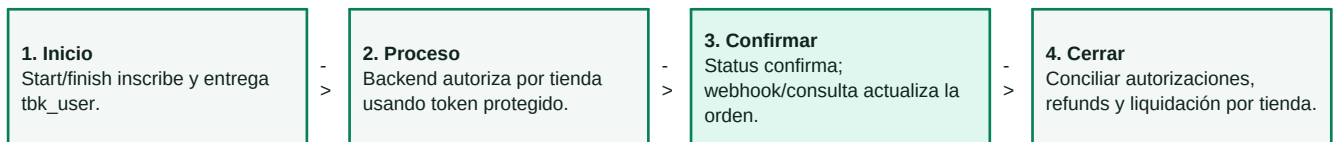
Sirve para: Cobros posteriores a una tarjeta previamente inscrita.

Modelo mental: Primero se inscribe la tarjeta y luego se cobra usando un token, nunca guardando sus datos completos.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Un cliente inscribe su tarjeta una vez y luego paga una compra de \$9.990. El comercio usa tbk_user, aplica límites y conserva el consentimiento de inscripción.	Start/finish inscribe y entrega tbk_user. Backend autoriza por tienda usando token protegido.	Status confirma; webhook/consulta actualiza la orden. Después: Conciliar autorizaciones, refunds y liquidación por tienda.

Figura 5. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Start/finish inscribe y entrega tbk_user.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Backend autoriza por tienda usando token protegido.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Status confirma; webhook/consulta actualiza la orden.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Conciliar autorizaciones, refunds y liquidación por tienda.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Baja o rechazo no permite reutilizar una inscripción inválida.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Completar start/finish de inscripción.
- Cifrar y asociar tbk_user al cliente.
- Autorizar con límites, status, refund y baja.

Evidencia de éxito: Inscripción terminada, autorización por tienda y consentimiento auditables.

Construcción concreta: Contrato Oneclick + REST + vault cifrado + consentimiento y límites.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	REST Oneclick: start/finish de inscripción; authorize/status/refund para cobros; delete para baja.
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Solicitar/contratar Oneclick Mall y usar primero sus credenciales y tarjetas de integración.

Costo: El DEMO cuesta \$0; afiliación y transacciones reales se cotizan/contratan con Transbank.

Decisión: Usa Transbank Oneclick Mall solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.
TRANSBANK_ONECLICK_COMMERCE_CODE	Sí	No	Identificar el comercio Oneclick.
TRANSBANK_ONECLICK_API_KEY	Sí	Sí	Autenticar la API Oneclick.
TRANSBANK_ONECLICK_BASE_URL	Sí	No	Separar explícitamente integración y producción.

Callbacks: /payments/oneclick/return

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.

- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- Transbank · API Oneclick
- Transbank · Cómo empezar

06. Khipu

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

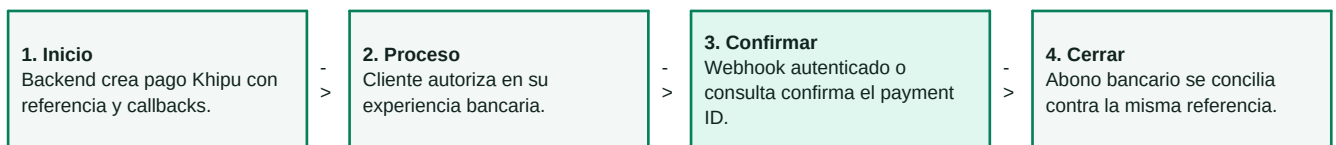
Sirve para: Transferencia cuenta a cuenta iniciada desde el comercio.

Modelo mental: El cliente autoriza una transferencia en la experiencia bancaria; tu comercio conserva referencia y estado.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Un cliente paga una factura de \$35.000 desde su banco. Khipu inicia la experiencia y el comercio espera un webhook o consulta del payment ID antes de entregar.	Backend crea pago Khipu con referencia y callbacks. Cliente autoriza en su experiencia bancaria.	Webhook autenticado o consulta confirma el payment ID. Después: Abono bancario se concilia contra la misma referencia.

Figura 6. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Backend crea pago Khipu con referencia y callbacks.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Cliente autoriza en su experiencia bancaria.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Webhook autenticado o consulta confirma el payment ID.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Abono bancario se concilia contra la misma referencia.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Callback perdido se recupera consultando; nunca por captura de pantalla.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Crear pago con retorno y notificación.
- Abrir la experiencia indicada por Khipu.
- Verificar webhook y consultar por payment_id.

Evidencia de éxito: Payment ID confirmado por API/evento y conciliado con el abono.

Construcción concreta: Cuenta cobrador + Payment API + webhook HTTPS + llave en secret manager.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	REST de pagos instantáneos: autenticar, crear pago, redirigir/abrir experiencia, consultar y recibir webhook.
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Registrarse como cobrador en Khipu, crear llaves/API y activar el ambiente indicado por el proveedor.

Costo: El DEMO cuesta \$0. Comisiones y liquidación varían por servicio/contrato; confirmar la tabla vigente con Khipu.

Decisión: Usa Khipu solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.
KHIPU_API_KEY	Sí	Sí	Autenticar la API de Khipu.

Callbacks: /webhooks/khipu; /payments/khipu/return

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- Khipu · Guía de implementación
- Khipu · Payment API

07. Mercado Pago

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

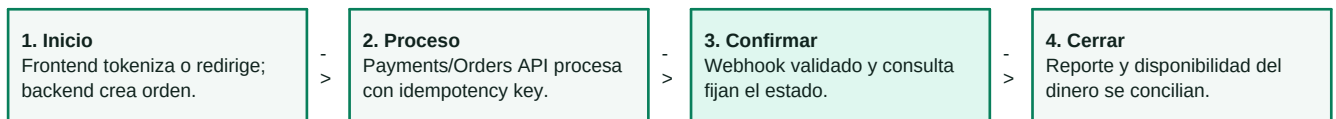
Sirve para: Checkout regional con tarjetas y otros medios.

Modelo mental: El frontend obtiene un token o abre un checkout; el access token y la creación viven en backend.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Una tienda cobra \$24.990 con Checkout. El frontend obtiene un token o redirección; el access token permanece en backend y el webhook se deduplica.	Frontend tokeniza o redirige; backend crea orden. Payments/Orders API procesa con idempotency key.	Webhook validado y consulta fijan el estado. Después: Reporte y disponibilidad del dinero se concilian.

Figura 7. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Frontend tokeniza o redirige; backend crea orden.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Payments/Orders API procesa con idempotency key.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Webhook validado y consulta fijan el estado.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Reporte y disponibilidad del dinero se concilian.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Evento duplicado se reconoce sin repetir ledger ni entrega.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Crear aplicación y credenciales de prueba.
- Tokenizar/iniciar en frontend sin exponer secreto.
- Crear con idempotencia, recibir webhook y consultar.

Evidencia de éxito: Payment/order ID con estado aprobado, webhook validado y movimiento conciliado.

Construcción concreta: Cuenta vendedor + app test + SDK frontend + REST backend + webhooks.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.

Capa	Tecnología / responsabilidad
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	REST/JSON Payments API existente; evaluar Orders API para una integración nueva. Tokenización en frontend y cobro en backend.
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Crear cuenta de vendedor y una aplicación en Mercado Pago Developers; separar credenciales de prueba y producción.

Costo: El DEMO cuesta \$0. La comisión real cambia por país, producto y plazo de disponibilidad: usar el enlace de costos de la cuenta.

Decisión: Usa Mercado Pago solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.
MERCADOPAGO_ACCESS_TOKEN	Sí	Sí	Autenticar llamadas servidor a servidor.

Callbacks: /webhooks/mercado-pago; /payments/mercado-pago/return

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- Mercado Pago · Primeros pasos
- Mercado Pago · Payments API

08. POS, mPOS y SoftPOS

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

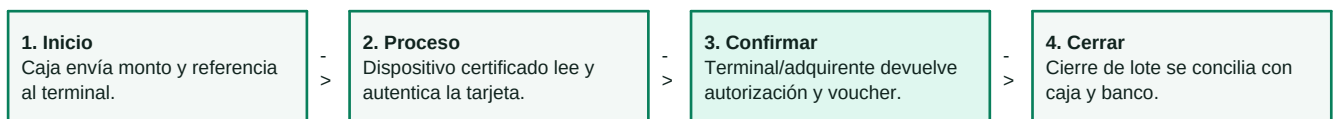
Sirve para: Cobro presencial en POS, mPOS o SoftPOS.

Modelo mental: El dispositivo certificado lee y autentica; la caja coordina el monto y espera el resultado.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Una caja envía \$18.500 a un POS. El terminal procesa la tarjeta y el cierre de lote debe coincidir con el voucher, la caja y el adquirente.	Caja envía monto y referencia al terminal. Dispositivo certificado lee y autentica la tarjeta.	Terminal/adquirente devuelve autorización y voucher. Después: Cierre de lote se concilia con caja y banco.

Figura 8. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapa	Qué ocurre	Pregunta de control
1. Inicio	Caja envía monto y referencia al terminal.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Dispositivo certificado lee y autentica la tarjeta.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Terminal/adquirente devuelve autorización y voucher.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Cierre de lote se concilia con caja y banco.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Desconexión exige consulta/reversa; no repetir venta a ciegas.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Contratar/adquirir dispositivo certificado.
- Integrar protocolo o SDK sin capturar PIN/PAN.
- Cerrar lote y conciliar terminal, adquirente y caja.

Evidencia de éxito: Voucher, identificador de terminal, autorización y cierre de lote coincidentes.

Construcción concreta: Terminal certificado + SDK/protocolo de caja + gestión de dispositivos.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.

Capa	Tecnología / responsabilidad
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa POS, mPOS y SoftPOS solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- PCI SSC · PCI DSS

- EMVCo · Especificaciones

09. Wallets tokenizadas

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

Sirve para: Apple Pay, Google Pay u otra wallet tokenizada.

Modelo mental: La wallet entrega un token de red y un criptograma de esa compra, no el número real de tarjeta.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Una persona paga \$32.990 con una wallet. El comercio recibe un token de red y criptograma, nunca el PAN completo, y procesa el cargo mediante su PSP.	Usuario elige wallet en dominio/app registrado. Wallet crea token de red y criptograma.	Backend/PSP valida token y confirma autorización. Después: Liquidación y disputa siguen el rail de tarjeta.

Figura 9. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapa	Qué ocurre	Pregunta de control
1. Inicio	Usuario elige wallet en dominio/app registrado.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Wallet crea token de red y criptograma.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Backend/PSP valida token y confirma autorización.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Liquidación y disputa siguen el rail de tarjeta.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Token inválido expira; fallback no debe exponer PAN.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Registrar dominio/app con wallet y PSP.
- Validar token/criptograma en backend o PSP.
- Procesar y conciliar como tarjeta tokenizada.

Evidencia de éxito: Token/criptograma aceptado, autorización y referencia de red.

Construcción concreta: SDK wallet + validación de dominio + PSP + gestión de claves.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.

Capa	Tecnología / responsabilidad
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Wallets tokenizadas solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- PCI SSC · PCI DSS
- EMVCo · Especificaciones

10. Saldo almacenado y gift cards

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

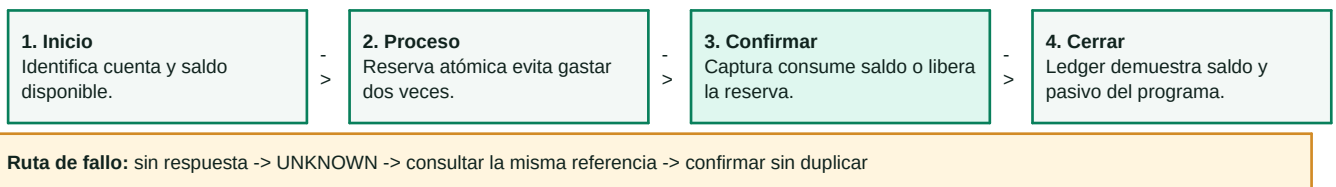
Sirve para: Gift card o saldo mantenido por el propio programa.

Modelo mental: El programa mantiene un saldo propio que debe reservarse y debitarse atómicamente.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Un cliente usa \$8.000 de una gift card con saldo \$10.000. El sistema reserva, captura y deja \$2.000 sin permitir dos consumos simultáneos.	Identifica cuenta y saldo disponible. Reserva atómica evita gastar dos veces.	Captura consume saldo o libera la reserva. Después: Ledger demuestra saldo y pasivo del programa.

Figura 10. Flujo verificable del caso



Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Identifica cuenta y saldo disponible.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Reserva atómica evita gastar dos veces.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Captura consume saldo o libera la reserva.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Ledger demuestra saldo y pasivo del programa.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Carrera o timeout se resuelve por idempotencia y reserva expirable.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Modelar cuenta y reglas de expiración.
- Reservar saldo con concurrencia segura.
- Capturar, liberar o restituir mediante asientos.

Evidencia de éxito: Saldo anterior + movimientos = saldo nuevo, con journal balanceado.

Construcción concreta: API propia + PostgreSQL transaccional + ledger + reglas regulatorias.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Saldo almacenado y gift cards solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- PCI SSC · PCI DSS
- EMVCo · Especificaciones

11. Mobile money

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

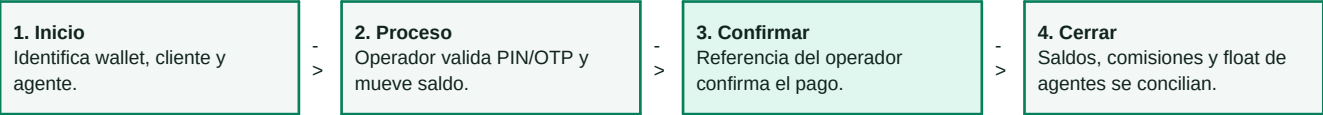
Sirve para: Pago desde saldo móvil con red de agentes.

Modelo mental: El valor vive en una cuenta móvil y entra o sale mediante agentes o bancos.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Una persona envía el equivalente a \$15.000 desde su wallet móvil a un comercio. El operador confirma y luego se cuadran saldos y comisión del agente.	Identifica wallet, cliente y agente. Operador valida PIN/OTP y mueve saldo.	Referencia del operador confirma el pago. Después: Saldos, comisiones y float de agentes se concilian.

Figura 11. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Identifica wallet, cliente y agente.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Operador valida PIN/OTP y mueve saldo.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Referencia del operador confirma el pago.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Saldos, comisiones y float de agentes se concilian.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Falta de liquidez o reversa abre una excepción trazable.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Integrar cuenta/identidad del operador.
- Implementar cash-in, pago y cash-out idempotentes.
- Conciliar float de agentes y settlement.

Evidencia de éxito: Referencia del operador y saldos de cliente, agente y tesorería cuadrados.

Construcción concreta: Contrato operador + API/USSD + KYC + gestión de agentes y float.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Mobile money solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- OWASP · Application Security Verification Standard
- PCI SSC · PCI DSS

12. Pagos QR

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

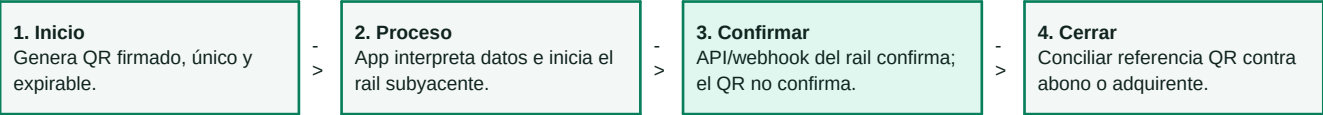
Sirve para: Iniciar un pago escaneando un código.

Modelo mental: El QR transporta datos para iniciar; por sí solo no demuestra que exista pago.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Un restaurante genera un QR por \$27.500 con expiración. Escanearlo sólo inicia el pago; el pedido se confirma cuando responde el rail subyacente.	Genera QR firmado, único y expirable. App interpreta datos e inicia el rail subyacente.	API/webhook del rail confirma; el QR no confirma. Después: Conciliar referencia QR contra abono o adquirente.

Figura 12. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Genera QR firmado, único y expirable.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	App interpreta datos e inicia el rail subyacente.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	API/webhook del rail confirma; el QR no confirma.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Conciliar referencia QR contra abono o adquirente.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: QR sustituido/expirado se rechaza; duplicado no reaplica.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Elegir QR estático o dinámico.
- Firmar/validar referencia, monto y expiración.
- Confirmar por rail y no por la pantalla del pagador.

Evidencia de éxito: Referencia QR única ligada a confirmación autoritativa.

Construcción concreta: Estándar QR + firma + backend + proveedor bancario o wallet.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Pagos QR solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- PCI SSC · PCI DSS
- EMVCo · Especificaciones

13. Transferencia bancaria

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

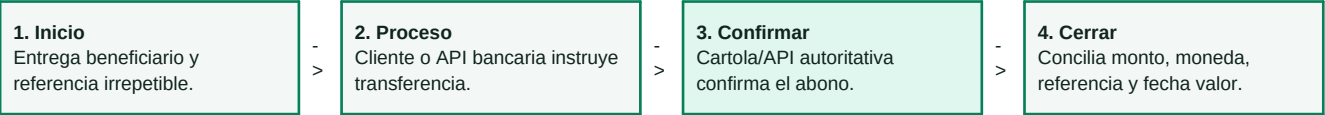
Sirve para: Pago directo entre cuentas bancarias.

Modelo mental: Una instrucción sale de una cuenta y llega a otra; el comprobante visual puede ser falso.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Un cliente transfiere \$180.000 por una factura. La imagen del comprobante no sirve como confirmación: el abono debe aparecer en la API o cartola bancaria.	Entrega beneficiario y referencia irrepitable. Cliente o API bancaria instruye transferencia.	Cartola/API autoritativa confirma el abono. Después: Concilia monto, moneda, referencia y fecha valor.

Figura 13. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Entrega beneficiario y referencia irrepitable.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Cliente o API bancaria instruye transferencia.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Cartola/API autoritativa confirma el abono.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Concilia monto, moneda, referencia y fecha valor.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Comprobante visual se ignora; diferencia queda pendiente.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Validar beneficiario y referencia.
- Iniciar o esperar transferencia por API/banco.
- Confirmar por cartola/API y conciliar.

Evidencia de éxito: Movimiento bancario autoritativo con cuenta, monto, moneda y referencia.

Construcción concreta: Cuenta empresa + API/open banking o archivos + conciliador.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Transferencia bancaria solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- ISO · ISO 20022
- BIS CPMI · sistemas de pago

14. ACH y transferencias batch

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

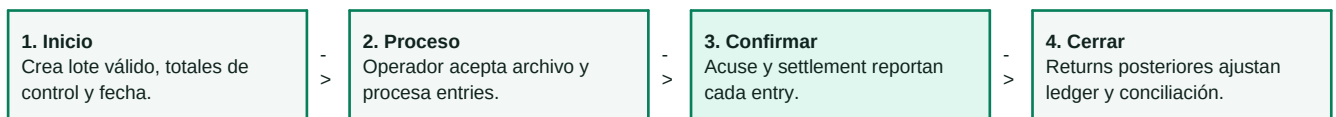
Sirve para: Transferencias masivas diferidas por lote.

Modelo mental: Las instrucciones viajan por lote y pueden liquidarse o devolverse días después.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Una empresa envía una nómina de 500 pagos. El operador acepta el lote hoy, pero cada entry puede liquidar o volver días después con su propio código.	Crea lote válido, totales de control y fecha. Operador acepta archivo y procesa entries.	Acuse y settlement reportan cada entry. Después: Returns posteriores ajustan ledger y conciliación.

Figura 14. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Crea lote válido, totales de control y fecha.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Operador acepta archivo y procesa entries.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Acuse y settlement reportan cada entry.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Returns posteriores ajustan ledger y conciliación.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Lote rechazado se corrige completo; return no se oculta.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Generar/validar archivo o usar API del operador.
- Controlar ventanas, lotes y totales.
- Procesar settlement y returns sin borrar el original.

Evidencia de éxito: Acuse del lote, settlement y return codes conciliados por entry.

Construcción concreta: Acuerdo ODFI/operador + formato NACHA/local + SFTP/API + scheduler.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa ACH y transferencias batch solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- ISO · ISO 20022
- BIS CPMI · sistemas de pago

15. Débito directo

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

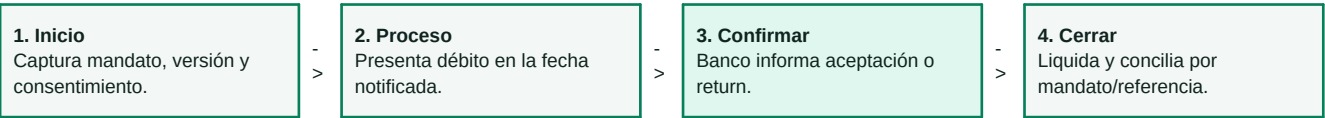
Sirve para: Cobros recurrentes autorizados desde cuenta.

Modelo mental: El comercio cobra porque existe un mandato previo, demostrable y revocable.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Una academia cobra \$29.990 mensuales. Antes del primer débito guarda el mandato; si se revoca o vuelve por fondos, detiene la secuencia según la regla.	Captura mandato, versión y consentimiento. Presenta débito en la fecha notificada.	Banco informa aceptación o return. Después: Liquida y concilia por mandato/referencia.

Figura 15. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Captura mandato, versión y consentimiento.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Presenta débito en la fecha notificada.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Banco informa aceptación o return.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Liquida y concilia por mandato/referencia.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Revocación o fondo insuficiente detiene nuevos intentos según regla.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Capturar consentimiento y versión del mandato.
- Presentar débitos con referencia única.
- Gestionar rechazos, cancelación y devoluciones.

Evidencia de éxito: Mandato vigente + presentación aceptada + settlement sin return.

Construcción concreta: Proveedor DD + vault de mandatos + batch/API + calendario y avisos.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Débito directo solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- ISO · ISO 20022
- BIS CPMI · sistemas de pago

16. Pagos instantáneos

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

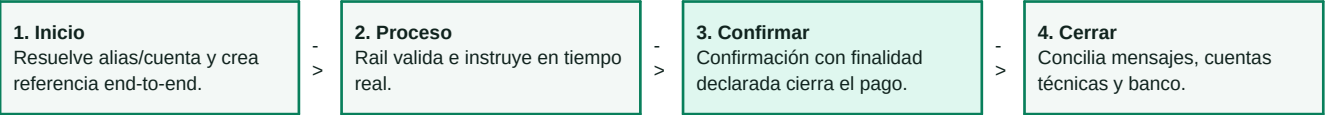
Sirve para: Transferencia cuenta a cuenta casi inmediata 24/7.

Modelo mental: El rail confirma en segundos, pero tu sistema aún necesita idempotencia y conciliación.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Un cliente paga \$42.000 mediante un rail 24/7. La respuesta llega en segundos, pero un timeout sigue necesitando consulta por referencia end-to-end.	Resuelve alias/cuenta y crea referencia end-to-end. Rail valida e instruye en tiempo real.	Confirmación con finalidad declarada cierra el pago. Después: Concilia mensajes, cuentas técnicas y banco.

Figura 16. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Resuelve alias/cuenta y crea referencia end-to-end.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Rail valida e instruye en tiempo real.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Confirmación con finalidad declarada cierra el pago.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Concilia mensajes, cuentas técnicas y banco.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: UNKNOWN exige status; rapidez no autoriza duplicar.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Resolver alias/beneficiario.
- Enviar instrucción o request-to-pay.
- Consumir confirmación final y conciliar en tiempo casi real.

Evidencia de éxito: ID end-to-end con finalidad confirmada por el rail.

Construcción concreta: Participante/proveedor + API ISO 20022/local + operación 24/7.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Pagos instantáneos solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- ISO · ISO 20022
- BIS CPMI · sistemas de pago

17. Links y solicitud de pago

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

Sirve para: Cobrar mediante link o solicitud compartible.

Modelo mental: Un link o solicitud conecta una orden con una experiencia de pago; no es el rail que liquida.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Un profesional envía un link por una factura de \$75.000. La URL expira y sólo conecta la orden con el medio elegido; no demuestra el pago por sí sola.	Crea orden firmada, expirable y de un solo uso. Cliente abre link y elige el rail final.	Backend confirma por API/webhook del rail. Después: Atribuye liquidación a orden, campaña o factura.

Figura 17. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Crea orden firmada, expirable y de un solo uso.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Cliente abre link y elige el rail final.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Backend confirma por API/webhook del rail.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Atribuye liquidación a orden, campaña o factura.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Link reutilizado o phishing se bloquea por firma/dominio.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Crear orden con expiración.
- Firmar y distribuir URL/QR.
- Atribuir el resultado del rail a la orden exacta.

Evidencia de éxito: Orden, click/iniciación y confirmación unidos por una referencia no reutilizable.

Construcción concreta: Frontend de checkout + URL firmada + PSP + antifraude.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Links y solicitud de pago solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- OWASP · Application Security Verification Standard
- PCI SSC · PCI DSS

18. Voucher pagable en efectivo

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

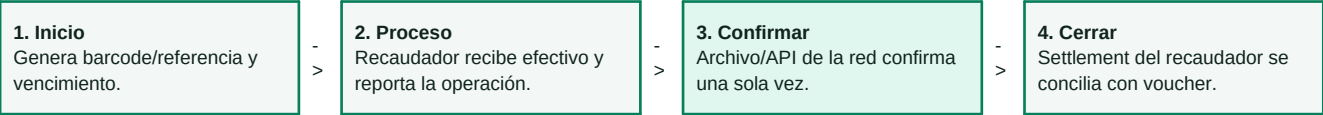
Sirve para: Orden digital pagada posteriormente en una red física.

Modelo mental: Una orden digital se paga físicamente; la confirmación llega después desde la red recaudadora.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Una tienda crea un voucher por \$16.990 pagable en un recaudador. La orden queda pendiente hasta recibir el archivo o evento de la red física.	Genera barcode/referencia y vencimiento. Recaudador recibe efectivo y reporta la operación.	Archivo/API de la red confirma una sola vez. Después: Settlement del recaudador se concilia con voucher.

Figura 18. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Genera barcode/referencia y vencimiento.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Recaudador recibe efectivo y reporta la operación.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Archivo/API de la red confirma una sola vez.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Settlement del recaudador se concilia con voucher.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Voucher vencido/duplicado se rechaza o queda en excepción.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Emitir barcode/referencia con expiración.
- Evitar reutilización y montos ambiguos.
- Procesar archivo/evento de recaudación.

Evidencia de éxito: Referencia pagada en tienda y settlement del recaudador.

Construcción concreta: Contrato recaudador + barcode + API/archivo + conciliación diferida.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Voucher pagable en efectivo solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- OWASP · Application Security Verification Standard
- PCI SSC · PCI DSS

19. BNPL y crédito alternativo

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

Sirve para: Compra financiada en cuotas por un tercero.

Modelo mental: Un financiador paga al comercio y el cliente adquiere una obligación en cuotas.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Una persona compra un teléfono en cuatro cuotas. El financiador evalúa y paga al comercio; el cliente acepta un contrato con costo total y calendario.	Solicita evaluación y muestra costo total. Cliente consiente; financiador aprueba y paga comercio.	API confirma contrato y desembolso. Después: Comercio concilia neto; financiador administra cuotas.

Figura 19. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Solicita evaluación y muestra costo total.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Cliente consiente; financiador aprueba y paga comercio.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	API confirma contrato y desembolso.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Comercio concilia neto; financiador administra cuotas.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Rechazo, mora o devolución siguen flujos separados.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Integrar evaluación y consentimiento.
- Mostrar costo total y calendario.
- Procesar desembolso, cuotas, refund y mora.

Evidencia de éxito: Contrato aceptado, desembolso al comercio y calendario contable.

Construcción concreta: Proveedor BNPL + API/SDK + consentimiento + regulación de crédito.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa BNPL y crédito alternativo solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- OWASP · Application Security Verification Standard
- PCI SSC · PCI DSS

20. Cobro mediante operador móvil

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

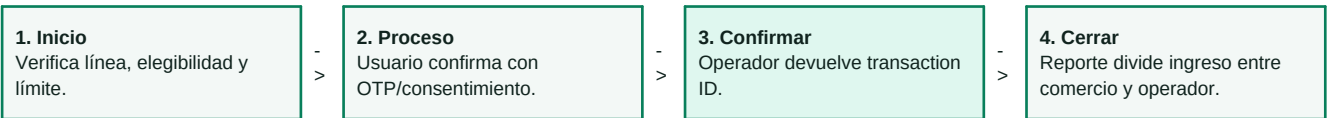
Sirve para: Cargo a factura o saldo del operador móvil.

Modelo mental: El operador añade el cargo a la factura o descuenta saldo móvil.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Un usuario compra contenido por \$3.990 cargado a su cuenta móvil. El operador valida línea, límite y OTP, y después reporta el reparto de ingresos.	Verifica línea, elegibilidad y límite. Usuario confirma con OTP/consentimiento.	Operador devuelve transaction ID. Después: Reporte divide ingreso entre comercio y operador.

Figura 20. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Verifica línea, elegibilidad y límite.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Usuario confirma con OTP/consentimiento.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Operador devuelve transaction ID.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Reporte divide ingreso entre comercio y operador.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Suscripción o cargo discutido se revierte con evidencia.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Validar número y elegibilidad.
- Obtener consentimiento/OTP y aplicar límites.
- Recibir resultado y conciliar settlement del operador.

Evidencia de éxito: Consentimiento, transaction ID y reporte del operador.

Construcción concreta: Contrato carrier/agregador + API + OTP + gestión de suscripciones.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Cobro mediante operador móvil solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- OWASP · Application Security Verification Standard
- PCI SSC · PCI DSS

21. Marketplaces, splits y payouts

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

Sirve para: Marketplace que cobra y paga a terceros.

Modelo mental: Un cobro bruto se divide entre comercio, plataforma, reserva y futuros payouts.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Un marketplace cobra \$100.000: \$85.000 son del vendedor, \$10.000 comisión y \$5.000 reserva. El subledger debe conservar esa igualdad.	Onboarding/KYB habilita cada beneficiario. Cobro crea split inmutable, comisión y reserva.	Webhook confirma entrada y subledger distribuye. Después: Payout liquida a vendedores y se concilia.

Figura 21. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Onboarding/KYB habilita cada beneficiario.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Cobro crea split inmutable, comisión y reserva.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Webhook confirma entrada y subledger distribuye.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Payout liquida a vendedores y se concilia.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Refund/chargeback reasigna saldos sin perder el bruto original.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Completar KYB/onboarding de participantes.
- Definir split inmutable por orden.
- Mantener subledger y ejecutar payouts conciliados.

Evidencia de éxito: Bruto = netos + comisión + reserva, y cada payout tiene trazabilidad.

Construcción concreta: PSP marketplace + cuentas conectadas + subledger + motor de payouts.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Marketplaces, splits y payouts solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- OWASP · Application Security Verification Standard
- PCI SSC · PCI DSS

22. Pagos B2B y tesorería

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

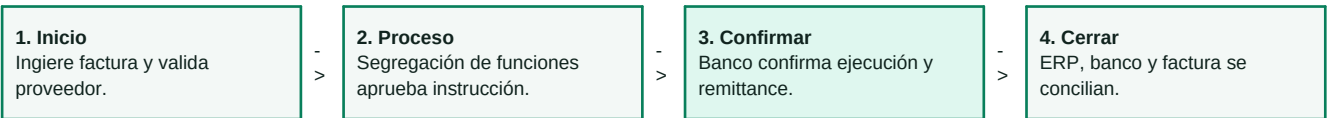
Sirve para: Pago de facturas con aprobaciones empresariales.

Modelo mental: La factura necesita aprobación, instrucción bancaria y aplicación contable en ERP.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Una compañía paga una factura de \$8.000.000. Compras valida, tesorería aprueba con doble control, el banco ejecuta y el ERP recibe la remittance.	Ingiere factura y valida proveedor. Segregación de funciones aprueba instrucción.	Banco confirma ejecución y remittance. Después: ERP, banco y factura se concilian.

Figura 22. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Ingiere factura y valida proveedor.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Segregación de funciones aprueba instrucción.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Banco confirma ejecución y remittance.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	ERP, banco y factura se concilian.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Duplicado/cambio de cuenta bloquea pago y escala revisión.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Ingerir y validar factura.
- Modelar segregación de funciones/aprobaciones.
- Enviar pago y aplicar remittance al ERP.

Evidencia de éxito: Factura aprobada, instrucción bancaria y asiento ERP con la misma referencia.

Construcción concreta: ERP + workflow de aprobación + API bancaria/SFTP + sanciones.

Capa	Tecnología / responsabilidad
Frontend	Interfaz web administrativa para registrar recepción, custodia y diferencias; no inicia una red de pagos.
Backend	Python/FastAPI o el stack transaccional existente del comercio.
API	API interna REST/JSON y, cuando exista, archivo/API de banco o red externa.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL con auditoría inmutable y ledger de doble entrada.
Operación	Control de acceso por rol, doble aprobación, respaldos y conciliación diaria.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Pagos B2B y tesorería solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- ISO · ISO 20022
- BIS CPMI · sistemas de pago

23. Pagos internacionales

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

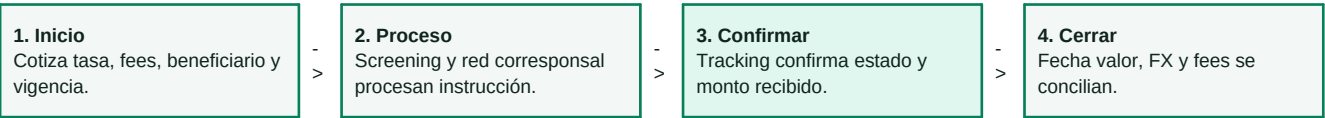
Sirve para: Pago entre países con FX y corresponsales.

Modelo mental: El valor cruza monedas y entidades; fees, FX, sanciones y fecha valor cambian el resultado.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Una empresa envía USD 10.000 a otro país. Antes fija tasa y fees; después explica cuánto salió, cuánto cobraron corresponsales y cuánto recibió el beneficiario.	Cotiza tasa, fees, beneficiario y vigencia. Screening y red corresponsal procesan instrucción.	Tracking confirma estado y monto recibido. Después: Fecha valor, FX y fees se concilian.

Figura 23. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Cotiza tasa, fees, beneficiario y vigencia.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Screening y red corresponsal procesan instrucción.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Tracking confirma estado y monto recibido.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Fecha valor, FX y fees se concilian.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Sanción, repair o devolución mantiene trazabilidad del original.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Cotizar FX y mostrar fees.
- Realizar screening y recopilar propósito.
- Rastrear corresponsales, settlement y monto recibido.

Evidencia de éxito: Referencia end-to-end, tasa/fees aceptados y confirmación del beneficiario.

Construcción concreta: Proveedor cross-border + KYC/AML + FX + SWIFT/ISO 20022.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Pagos internacionales solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- ISO · ISO 20022
- BIS CPMI · sistemas de pago

24. RTGS y alto valor

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

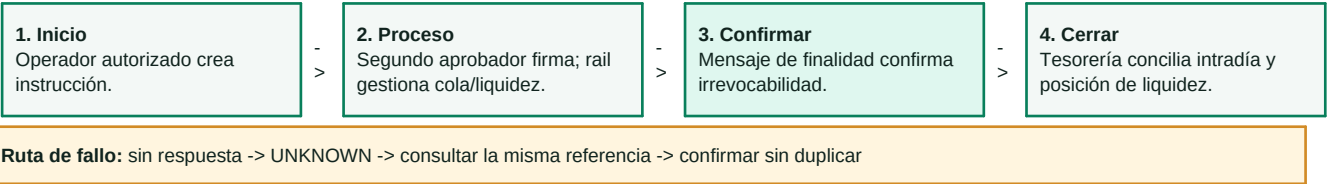
Sirve para: Transferencia urgente de alto valor por RTGS.

Modelo mental: RTGS prioriza finalidad y liquidez; una operación puede esperar en cola.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Tesorería instruye un pago RTGS de alto valor. Dos personas autorizan, el rail puede ponerlo en cola y sólo la finalidad oficial permite cerrarlo.	Operador autorizado crea instrucción. Segundo aprobador firma; rail gestiona cola/liquidez.	Mensaje de finalidad confirma irrevocabilidad. Después: Tesorería concilia intradía y posición de liquidez.

Figura 24. Flujo verificable del caso



Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Operador autorizado crea instrucción.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Segundo aprobador firma; rail gestiona cola/liquidez.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Mensaje de finalidad confirma irrevocabilidad.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Tesorería concilia intradía y posición de liquidez.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Rechazo/cola no se marca pagado; escala a operación crítica.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Integrar mensajería/canal autorizado.
- Controlar aprobación dual, prioridad y liquidez.
- Registrar aceptación final y conciliación intradía.

Evidencia de éxito: Mensaje aceptado por el sistema RTGS con finalidad y timestamp.

Construcción concreta: Acceso institucional + PKI/HSM + doble control + monitoreo RTGS.

Capa	Tecnología / responsabilidad
Frontend	Interfaz web administrativa para registrar recepción, custodia y diferencias; no inicia una red de pagos.
Backend	Python/FastAPI o el stack transaccional existente del comercio.
API	API interna REST/JSON y, cuando exista, archivo/API de banco o red externa.
Datos	PostgreSQL con auditoría inmutable y ledger de doble entrada.

Capa	Tecnología / responsabilidad
Operación	Control de acceso por rol, doble aprobación, respaldos y conciliación diaria.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa RTGS y alto valor solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- ISO · ISO 20022
- BIS CPML · sistemas de pago

25. Open Finance e iniciación

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

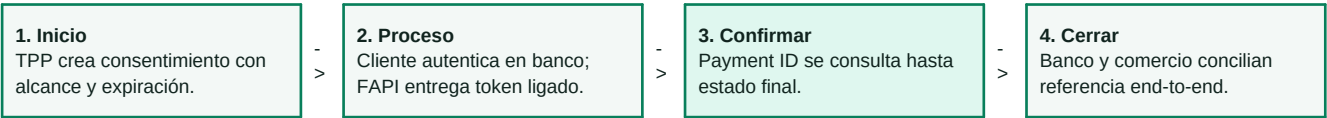
Sirve para: Iniciar pago desde una app con consentimiento bancario.

Modelo mental: Una aplicación inicia desde la cuenta solo dentro del consentimiento y alcance otorgados.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Una app inicia \$55.000 desde la cuenta bancaria del usuario. El consentimiento limita monto y propósito; FAPI y mTLS protegen el intercambio con el banco.	TPP crea consentimiento con alcance y expiración. Cliente autentica en banco; FAPI entrega token ligado.	Payment ID se consulta hasta estado final. Después: Banco y comercio concilian referencia end-to-end.

Figura 25. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	TPP crea consentimiento con alcance y expiración.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Cliente autentica en banco; FAPI entrega token ligado.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Payment ID se consulta hasta estado final.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Banco y comercio concilian referencia end-to-end.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Consentimiento revocado/token vencido exige nueva autorización.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Registrar TPP/cliente según regulación.
- Implementar FAPI, PKCE y consentimiento.
- Usar token ligado, iniciar y consultar el pago.

Evidencia de éxito: Consent ID, token con alcance mínimo y payment ID confirmado.

Construcción concreta: Registro/licencia TPP + OAuth/OIDC FAPI + mTLS + API bancaria.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Open Finance e iniciación solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- OpenID Foundation · FAPI
- IETF · OAuth 2.0 Security BCP

26. Bitcoin, Lightning y activos digitales

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

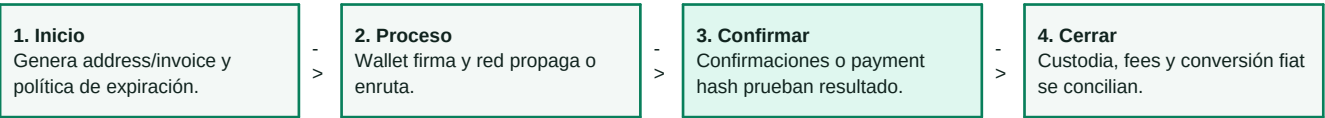
Sirve para: Transferir Bitcoin, Lightning u otro activo.

Modelo mental: Una firma autoriza transferencia del activo; confirmación y custodia definen el riesgo real.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Un comercio emite una invoice Lightning o dirección Bitcoin por un monto y tiempo definidos. Payment hash o confirmaciones se enlazan con la orden.	Genera address/invoice y política de expiración. Wallet firma y red propaga o enruta.	Confirmaciones o payment hash prueban resultado. Después: Custodia, fees y conversión fiat se concilian.

Figura 26. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etap	Qué ocurre	Pregunta de control
1. Inicio	Genera address/invoice y política de expiración.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Wallet firma y red propaga o enruta.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Confirmaciones o payment hash prueban resultado.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Custodia, fees y conversión fiat se concilian.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Reorg, fee insuficiente o pérdida de clave usa runbook específico.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Decidir custodia y red.
- Crear address/invoice y política de fees.
- Observar mempool/confirmaciones o settlement Lightning.

Evidencia de éxito: Transaction ID o payment hash validado y conciliado con la orden.

Construcción concreta: Nodo/custodio + wallet segura + indexador + compliance y recovery.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Bitcoin, Lightning y activos digitales solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- Bitcoin Developer Guides
- Lightning specifications

27. Pagos máquina a máquina

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

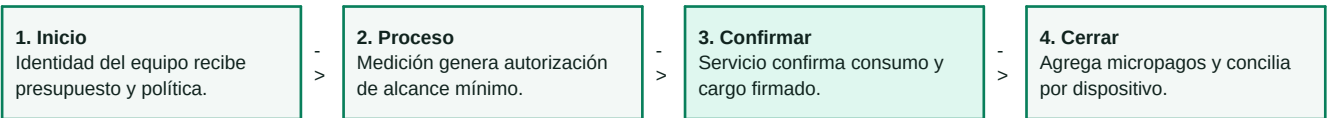
Sirve para: Dispositivo paga automáticamente por uso.

Modelo mental: Una máquina paga dentro de una política; no recibe autoridad ilimitada.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Un cargador de vehículo paga automáticamente por 18 kWh. La identidad del dispositivo, la medición y un presupuesto máximo deben viajar juntos.	Identidad del equipo recibe presupuesto y política. Medición genera autorización de alcance mínimo.	Servicio confirma consumo y cargo firmado. Después: Agrega micropagos y concilia por dispositivo.

Figura 27. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Identidad del equipo recibe presupuesto y política.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Medición genera autorización de alcance mínimo.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Servicio confirma consumo y cargo firmado.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Agrega micropagos y concilia por dispositivo.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Equipo comprometido se revoca; límite evita gasto ilimitado.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Identificar dispositivo y medidor.
- Emitir credencial limitada por monto/uso/tiempo.
- Autorizar, registrar consumo y conciliar.

Evidencia de éxito: Identidad, medición, decisión de política y settlement unidos.

Construcción concreta: Identidad de dispositivo + attestation + API + wallet/rail + límites.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.

Capa	Tecnología / responsabilidad
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Pagos máquina a máquina solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- Bitcoin Developer Guides
- Lightning specifications

28. Pagos realizados por agentes

[Volver al índice](#) | [Fuente Markdown](#) | [HTML público](#)

En una frase

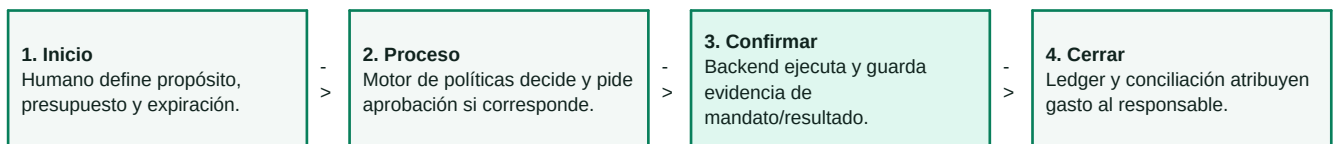
Sirve para: Un agente de software compra bajo mandato.

Modelo mental: El agente propone o ejecuta dentro de un mandato verificable y límites humanos.

Ejemplo concreto

Situación	Detrás de la pantalla	Se acepta como pagado cuando
Un agente reserva un hotel dentro de un presupuesto de \$300.000. Si cambia precio, destino o límite, pide aprobación humana antes de ejecutar.	Humano define propósito, presupuesto y expiración. Motor de políticas decide y pide aprobación si corresponde.	Backend ejecuta y guarda evidencia de mandato/resultados. Después: Ledger y conciliación atribuyen gasto al responsable.

Figura 28. Flujo verificable del caso



Ruta de fallo: sin respuesta -> UNKNOWN -> consultar la misma referencia -> confirmar sin duplicar

Recorrido de comienzo a fin

Etapas	Qué ocurre	Pregunta de control
1. Inicio	Humano define propósito, presupuesto y expiración.	¿Hay referencia, monto, moneda y expiración?
2. Proceso	Motor de políticas decide y pide aprobación si corresponde.	¿Qué sistema puede producir el efecto financiero?
3. Confirmación	Backend ejecuta y guarda evidencia de mandato/resultados.	¿La evidencia viene de una fuente autoritativa?
4. Cierre	Ledger y conciliación atribuyen gasto al responsable.	¿Ledger, proveedor y banco explican el resultado?

Qué ocurre si falla

Fallo característico: Ambigüedad o exceso de límite detiene, no improvisa.

Un timeout no equivale a rechazo. Conserva UNKNOWN, consulta con la misma referencia y no generes un segundo efecto a ciegas.

Cómo llevarlo a una aplicación real

- Definir mandato, presupuesto y comercios permitidos.
- Exigir aprobación humana por riesgo.
- Usar credencial de alcance mínimo y audit trail.

Evidencia de éxito: Mandato + decisión + aprobación + referencia del pago reproducibles.

Construcción concreta: Agent API + policy engine + aprobación humana + credencial limitada.

Capa	Tecnología / responsabilidad
Frontend	TypeScript + HTML/CSS. El navegador presenta el checkout y recibe el retorno; no guarda secretos.
Backend	Python 3.11+ con FastAPI para este laboratorio. Node.js/TypeScript, Java, .NET o Go son equivalentes si el equipo los opera mejor.

Capa	Tecnología / responsabilidad
API	HTTPS REST/JSON entre tu backend y el proveedor; webhooks HTTPS para cambios asíncronos.
Datos	PostgreSQL para intents, eventos, idempotencia y ledger; Redis solo para cache/locks, nunca como libro contable.
Operación	Docker, gestor de secretos, logs estructurados, métricas, alertas y job de conciliación.

Alta, costo y decisión

Dónde comenzar: Elegir un proveedor regulado que ofrezca esta modalidad en tu país, abrir cuenta comercial, verificar la empresa y solicitar sandbox.

Costo: El DEMO cuesta \$0. Precio, impuestos, reservas y plazo de liquidación dependen del proveedor, país y volumen; pedir cotización vigente.

Decisión: Usa Pagos realizados por agentes solo si su cobertura, experiencia, reversibilidad y conciliación resuelven tu caso; compara al menos dos proveedores antes de LIVE.

Variables y callbacks

Variable	Req.	Secreta	Propósito
PAYLAB_HOST	No	No	Interfaz local. Solo se aceptan direcciones loopback.
PAYLAB_PORT	No	No	Puerto del portal localhost.
PAYLAB_CONFIG_DIR	No	No	Directorio alternativo para catálogo y guías.

Callbacks: No aplica hasta elegir proveedor.

Seguridad

- El monto, moneda, beneficiario y referencia nacen o se validan en el backend; nunca se confía en el navegador.
- Secretos en un gestor de secretos, rotación y mínimo privilegio; jamás en JavaScript, Git o logs.
- TLS, validación de firma/origen de webhooks, deduplicación persistente e idempotency key por operación.
- Minimizar datos: almacenar tokens y referencias del proveedor, no PAN, CVV, claves ni credenciales bancarias.
- Cifrado en reposo, control de acceso, trazabilidad y política de retención/borrado.

Pruebas mínimas

- Happy path con montos mínimos, normales y límites documentados.
- Rechazo, cancelación del usuario, expiración y retorno del navegador manipulado.
- Timeout antes y después de enviar, webhook tardío, duplicado y fuera de orden.
- Refund total/parcial cuando aplique y conciliación con una diferencia intencional.
- Prueba end-to-end en sandbox/certificación con credenciales separadas; producción solo después de aprobar el checklist.

Checklist antes de LIVE

- Contrato y cuenta comercial aprobados; costos y plazos confirmados directamente con el proveedor.
- Credenciales productivas separadas, callbacks HTTPS públicos, dominios permitidos y rotación documentada.
- Alertas, runbook, soporte, refund/disputa y conciliación ensayados con responsables definidos.
- Piloto con límites y monitoreo reforzado; rollback que detiene nuevas operaciones sin perder evidencia.

Fuentes

- Bitcoin Developer Guides
- Lightning specifications